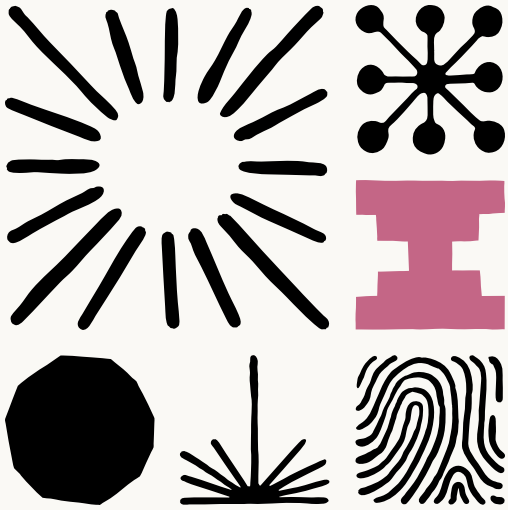
Engineering at Anthropic



Building a C compiler with a team of parallel Claudes

We tasked Opus 4.6 using agent teams to build a C Compiler, and then (mostly) walked away. Here's what it taught us about the future of autonomous software development.

Published Feb 05, 2026

Written by Nicholas Carlini, a researcher on our Safeguards team.

I've been experimenting with a new approach to supervising language models that we're calling "agent teams."

With agent teams, multiple Claude instances work in parallel on a shared codebase without active human intervention. This approach dramatically expands the scope of what's achievable with LLM agents.

To stress test it, I tasked 16 agents with writing a Rust-based C compiler, from scratch, capable of compiling the Linux kernel. Over nearly 2,000 Claude Code sessions and $20,000 in API costs, the agent team produced a 100,000-line compiler that can build Linux 6.9 on x86, ARM, and RISC-V.

The compiler is an interesting artifact on its own, but I focus here on what I learned about designing harnesses for long-running autonomous agent teams: how to write tests that keep agents on track without human oversight, how to structure work so multiple agents can make progress in parallel, and where this approach hits its ceiling.

Enabling long-running Claudes

Existing agent scaffolds like Claude Code require an operator to be online and available to work jointly. If you ask for a solution to a long and complex problem, the model may solve part of it, but eventually it will stop and wait for continued input—a question, a status update, or a request for clarification.

To elicit sustained, autonomous progress, I built a harness that sticks Claude in a simple loop (if you've seen Ralph-loop, this should look familiar). When it finishes one task, it immediately picks up the next. *(Run this in a container, not your actual machine).*

```
#!/bin/bash

while true; do
    COMMIT=$(git rev-parse --short=6 HEAD)
    LOGFILE="agent_logs/agent_${COMMIT}.log"

    claude --dangerously-skip-permissions \
           -p "$(cat AGENT_PROMPT.md)" \
           --model claude-opus-X-Y &> "$LOGFILE"
done
```

Copy

In the agent prompt, I tell Claude what problem to solve and ask it to approach the problem by breaking it into small pieces, tracking what it's working on, figuring out what to work on next, and to effectively keep going until it's perfect. (On this last point, Claude has no choice. The loop runs forever—although in one instance, I did see Claude `pkill -9 bash` on accident, thus killing itself and ending the loop. Whoops!).

Running Claude in parallel

Running multiple instances in parallel can address two weaknesses of a single-agent harness:

- One Claude Code session can only do one thing at a time. Especially as the scope of a project expands, debugging multiple issues in parallel is far more efficient.
- Running multiple Claude agents allows for specialization. While a few agents are tasked to solve the actual problem at hand, other specialized agents can be invoked to (for example) maintain documentation, keep an eye on code quality, or solve specialized sub-tasks.

My implementation of parallel Claude is bare-bones. A new bare git repo is created, and for each agent, a Docker container is spun up with the repo mounted to `/upstream`. Each agent clones a local copy to `/workspace`, and when it's done, pushes from its own local container to upstream.

To prevent two agents from trying to solve the same problem at the same time, the harness uses a simple synchronization algorithm:

1. Claude takes a "lock" on a task by writing a text file to current_tasks/ (e.g., one agent might lock current_tasks/parse_if_statement.txt, while another locks current_tasks/codegen_function_definition.txt). If two agents try to claim the same task, git's synchronization forces the second agent to pick a different one.
2. Claude works on the task, then pulls from upstream, merges changes from other agents, pushes its changes, and removes the lock. Merge conflicts are frequent, but Claude is smart enough to figure that out.
3. The infinite agent-generation-loop spawns a new Claude Code session in a fresh container, and the cycle repeats.

This is a very early research prototype. I haven't yet implemented any other method for communication between agents, nor do I enforce any process for managing high-level goals. I don't use an orchestration agent.

Instead, I leave it up to each Claude agent to decide how to act. In most cases, Claude picks up the "next most obvious" problem. When stuck on a bug, Claude will often maintain a running doc of failed approaches and remaining tasks. In the git repository of the project, you can read through the history and watch it take out locks on various tasks.

Lessons from programming with Claude agent teams

The scaffolding runs Claude in a loop, but that loop is only useful if Claude can tell how to make progress. Most of my effort went into designing the environment around

Claude—the tests, the environment, the feedback—so that it could orient itself without me. These are the approaches I've found most helpful when orchestrating multiple Claude instances.

Write extremely high-quality tests

Claude will work autonomously to solve whatever problem I give it. So it's important that the task verifier is nearly perfect, otherwise Claude will solve the wrong problem. Improving the testing harness required finding high-quality compiler test suites, writing verifiers and build scripts for open-source software packages, and watching for mistakes Claude was making, then designing new tests as I identified those failure modes.

For example, near the end of the project, Claude started to frequently break existing functionality each time it implemented a new feature. To address this, I built a continuous integration pipeline and implemented stricter enforcement that allowed Claude to better test its work so that new commits can't break existing code.

Put yourself in Claude's shoes

I had to constantly remind myself that I was writing this test harness for Claude and not for myself, which meant rethinking many of my assumptions about how tests should communicate results.

For example, each agent is dropped into a fresh container with no context and will spend significant time orienting itself, especially on large projects. Before we even reach the tests, to help Claude help itself, I included instructions to maintain extensive READMEs and progress files that should be updated frequently with the current status.

I also kept in mind the fact that language models have inherent limitations, which, in this case, needed to be designed around. These include:

- **Context window pollution:** The test harness should not print thousands of useless bytes. At most, it should print a few lines of output and log all important information to a file so Claude can find it when needed. Logfiles should be easy to process automatically: if there are errors, Claude should write ERROR and put the reason on the same line so grep will find it. It helps to pre-compute aggregate summary statistics so Claude doesn't have to recompute them.

- **Time blindness:** Claude can't tell time and, left alone, will happily spend hours running tests instead of making progress. The harness prints incremental progress infrequently (to avoid polluting context) and includes a default `--fast` option that runs a 1% or 10% random sample. This subsample is deterministic per-agent but random across VMs, so Claude still covers all files but each agent can perfectly identify regressions.

Make parallelism easy

When there are many distinct failing tests, parallelization is trivial: each agent picks a different failing test to work on. After the test suite reached a 99% pass rate, each agent worked on getting a different small open-source project (e.g., SQlite, Redis, libjpeg, MQuickJS, Lua) to compile.

But when agents started to compile the Linux kernel, they got stuck. Unlike a test suite with hundreds of independent tests, compiling the Linux kernel is one giant task. Every agent would hit the same bug, fix that bug, and then overwrite each other's changes. Having 16 agents running didn't help because each was stuck solving the same task.

The fix was to use GCC as an online known-good compiler oracle to compare against. I wrote a new test harness that randomly compiled most of the kernel using GCC, and only the remaining files with Claude's C Compiler. If the kernel worked, then the problem wasn't in Claude's subset of the files. If it broke, then it could further refine by re-compiling some of these files with GCC. This let each agent work in parallel, fixing different bugs in different files, until Claude's compiler could eventually compile all files. (After this worked, it was still necessary to apply delta debugging techniques to find pairs of files that failed together but worked independently.)

Multiple agent roles

Parallelism also enables specialization. LLM-written code frequently re-implements existing functionality, so I tasked one agent with coalescing any duplicate code it found. I put another in charge of improving the performance of the compiler itself, and a third I made responsible for outputting efficient compiled code. I asked another agent to critique the design of the project from the perspective of a Rust developer, and make structural changes to the project to improve the overall code quality, and another to work on documentation.

Stress testing the limits of agent teams

This project was designed as a capability benchmark. I am interested in stress-testing the limits of what LLMs can just *barely* achieve today in order to help us prepare for what models will reliably achieve in the future.

I've been using the C Compiler project as a benchmark across the entire Claude 4 model series. As I did with prior projects, I started by drafting what I wanted: a from-scratch optimizing compiler with no dependencies, GCC-compatible, able to compile the Linux kernel, and designed to support multiple backends. While I specified some aspects of the design (e.g., that it should have an SSA IR to enable multiple optimization passes) I did not go into any detail on how to do so.

Previous Opus 4 models were barely capable of producing a functional compiler. Opus 4.5 was the first to cross a threshold that allowed it to produce a functional compiler which could pass large test suites, but it was still incapable of compiling any real large projects. My goal with Opus 4.6 was to again test the limits.

Evaluation

Over nearly 2,000 Claude Code sessions across two weeks, Opus 4.6 consumed 2 billion input tokens and generated 140 million output tokens, a total cost just under $20,000. Compared to even the most expensive Claude Max plans, this was an extremely expensive project. But that total is a fraction of what it would cost me to produce this myself—let alone an entire team.

This was a clean-room implementation (Claude did not have internet access at any point during its development); it depends only on the Rust standard library. The 100,000-line compiler can build a bootable Linux 6.9 on x86, ARM, and RISC-V. It can also compile QEMU, FFmpeg, SQlite, postgres, redis, and has a 99% pass rate on most compiler test suites including the GCC torture test suite. It also passes the developer's ultimate litmus test: it can compile and run Doom.

The compiler, however, is not without limitations. These include:

- It lacks the 16-bit x86 compiler that is necessary to boot Linux out of real mode. For this, it calls out to GCC (the x86_32 and x86_64 compilers are its own).
- It does not have its own assembler and linker; these are the very last bits that Claude started automating and are still somewhat buggy. The demo video was produced with a GCC assembler and linker.

- The compiler successfully builds many projects, but not all. It's not yet a drop-in replacement for a real compiler.
- The generated code is not very efficient. Even with all optimizations enabled, it outputs less efficient code than GCC with all optimizations *disabled*.
- The Rust code quality is reasonable, but is nowhere near the quality of what an expert Rust programmer might produce.

The resulting compiler has nearly reached the limits of Opus's abilities. I tried (hard!) to fix several of the above limitations but wasn't fully successful. New features and bugfixes frequently broke existing functionality.

As one particularly challenging example, Opus was unable to implement a 16-bit x86 code generator needed to boot into 16-bit real mode. While the compiler can output correct 16-bit x86 via the 66/67 opcode prefixes, the resulting compiled output is over 60kb, far exceeding the 32k code limit enforced by Linux. Instead, Claude simply cheats here and calls out to GCC for this phase (This is only the case for x86. For ARM or RISC-V, Claude's compiler can compile completely by itself.)

The source code for the compiler is available. Download it, read through the code, and try it on your favorite C projects. I've consistently found the best way to understand what language models can do is to push them to their limits, and then study where they start to break down. Over the coming days, I'll continue having Claude push new changes if you want to follow along with Claude's continued attempts at addressing these limitations.

Looking forward

Each generation of language models opens up new ways of working with them. Early models were useful for tab-completion in IDEs. Before long, models could complete a function body from its docstring. The launch of Claude Code brought agents into the mainstream and enabled developers to pair-program with Claude. But each of these products operates under the assumption that a user defines a task, an LLM runs for a few seconds or minutes and returns an answer, and then the user provides a follow-up.

Agent teams show the possibility of implementing entire, complex projects autonomously. This allows us, as users of these tools, to become more ambitious with our goals.

We are still early, and fully autonomous development comes with real risks. When a human sits with Claude during development, they can ensure consistent quality and catch errors in real time. For autonomous systems, it is easy to see tests pass and assume the job is done, when this is rarely the case. I used to work in penetration testing, exploiting vulnerabilities in products produced by large companies, and the thought of programmers deploying software they've never personally verified is a real concern.

So, while this experiment excites me, it also leaves me feeling uneasy. Building this compiler has been some of the most fun I've had recently, but I did not expect this to be anywhere near possible so early in 2026. The rapid progress in both language models and the scaffolds we use to interact with them opens the door to writing an enormous amount of new code. I expect the positive applications to outweigh the negative, but we're entering a new world which will require new strategies to navigate safely.

Acknowledgements

Special thanks to Josef Bacik, Edwin Chen, Bernardo Meurer Costa, Jake Eaton, Dan Kelley, Felix Klock, Jannet Park, Steve Weis, and many other people across Anthropic for their assistance and contributions.

Products

Claude

Claude Code

Claude Code Enterprise

Claude Cowork

@Claude

Claude Design

Claude Science

Claude Security

Claude for Chrome

Claude for Microsoft 365

Skills

Download app

Pricing

Log in to Claude

Models

Mythos

Fable

Opus

Sonnet

Haiku

Solutions

AI agents

Code modernization

Coding

Customer support

Education

Enterprise

Financial services

Government

Healthcare

Legal

Life sciences

Nonprofits

Security

Small business

Claude Platform

Overview

Developer docs

Pricing

Ecosystem

Marketplace

Regional compliance

Claude on AWS

Google Cloud

Microsoft Foundry

Console login

Resources

Blog

Claude partner network

Community

Connectors

Courses

Customer stories

Engineering at Anthropic

Events

Plugins

Powered by Claude

Service partners

Tutorials

Use cases

Programs

Startups

Research Labs

Help and security

Availability

Status

Support center

Company

Anthropic

Careers

Policy

Economic Futures

Research

News

Claude's Constitution

Claude Corps

Keep thinking

Policy on the AI Exponential

Responsible Scaling Policy

Security and compliance

Transparency

Terms and policies

Privacy policy

Consumer health data privacy policy

Responsible disclosure policy

Terms of service: Commercial

Terms of service: Consumer

Usage policy